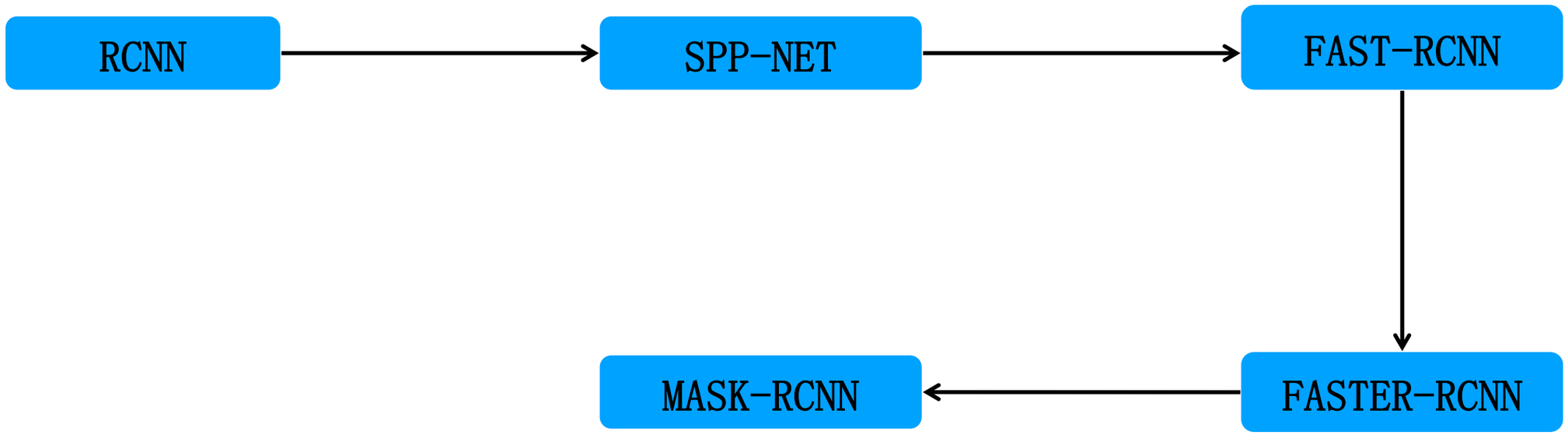
江南大学
JIANGNAN UNIVERSITY

Two-Stage Detector

马栋梁
2018-05-10

Context



[1]Girshick R B, Donahue J, Darrell T, et al. Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation[J]. computer vision and pattern recognition, 2014: 580-587.
[2]K. He, X. Zhang, S. Ren, and J. Sun, “Spatial pyramid pooling in deep convolutional networks for visual recognition,” in European Conference on Computer Vision (ECCV), 2014.
[3]R. Girshick, “Fast R-CNN,” in IEEE International Conference on Computer Vision (ICCV), 2015.
[4]Ren S, He K, Girshick R B, et al. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2017, 39(6): 1137-1149.

RCNN

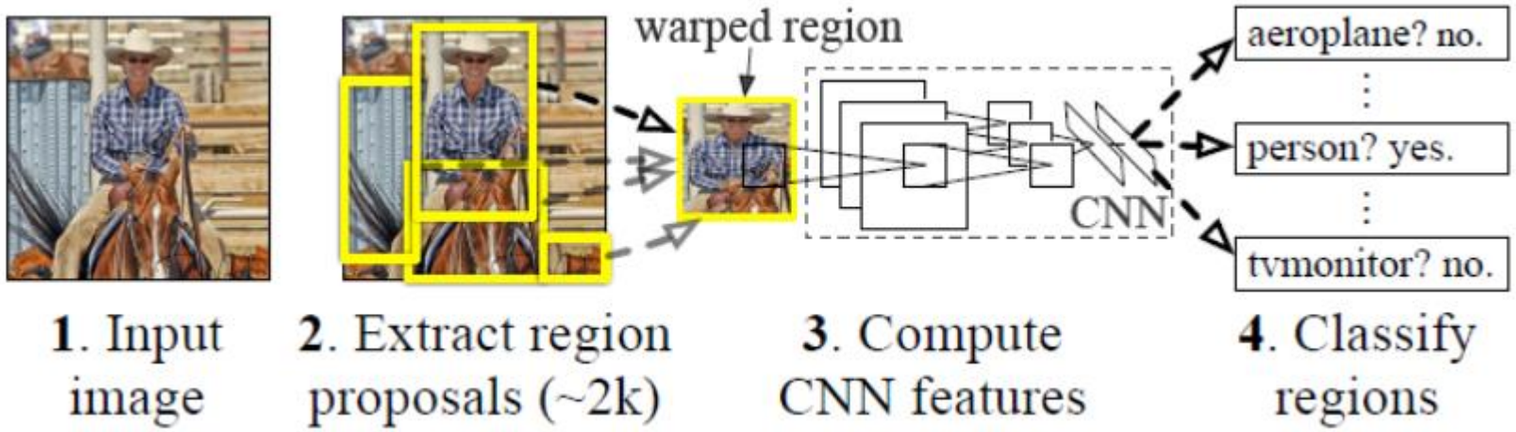
Q:是否可以采用CNN特征来提高当前一直停滞不前的物体检测准确率?

&创新点:

(1)采用CNN网络提取图像特征，从经验驱动的人造特征范式HOG、SIFT到数据驱动的表示学习范式，提高特征对样本的表示能力。

(2)采用大样本下有监督预训练+小样本微调的方式解决小样本难以训练甚至过拟合等问题。

RCNN-network



1. selective-search
2. coordinate regression

相关链接：https://blog.csdn.net/wopawn/article/details/52133338

RCNN-selective search(2012 IJCV)

算法简要步骤:

1. 使用 Efficient Graph-Based Image Segmentation的方法获取原始分割区域R={r1,r2,…,rn}
2. 初始化相似度集合S=∅
3. 计算两两相邻区域之间的相似度，将其添加到相似度集合S中。
4. 从相似度集合S中找出，相似度最大的两个区域 ri 和rj，将其合并成为一个区域 rt，从相似度集合中除去原先与ri和rj相邻区域之间计算的相似度，计算rt与其相邻区域（原先与ri或rj相邻的区域）的相似度，将其结果添加的到相似度集合S中。同时将新区域 rt 添加到 区域集合R中。
5. 获取每个区域的Bounding Boxes，这个结果就是物体位置的可能结果L.

相关链接：https://blog.csdn.net/mao_kun/article/details/50576003

RCNN-selective search

&Efficient Graph-Based Image Segmentation（2004 IJCV）：

Algorithm 1 *Segmentation algorithm.*

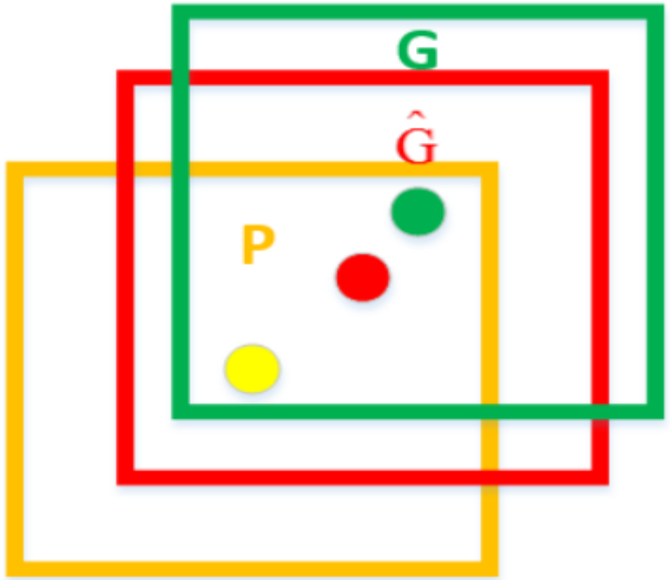
The input is a graph $G = (V, E)$, with n vertices and m edges. The output is a segmentation of V into components $S = (C_1, \ldots, C_r)$.

0. Sort E into $\pi = (o_1, \ldots, o_m)$, by non-decreasing edge weight.
1. Start with a segmentation S^0, where each vertex v_i is in its own component.
2. Repeat step 3 for $q = 1, \ldots, m$.
3. Construct S^q given S^{q-1} as follows. Let v_i and v_j denote the vertices connected by the q-th edge in the ordering, i.e., $o_q = (v_i, v_j)$. If v_i and v_j are in disjoint components of S^{q-1} and $w(o_q)$ is small compared to the internal difference of both those components, then merge the two components otherwise do nothing. More formally, let C_i^{q-1} be the component of S^{q-1} containing v_i and C_j^{q-1} the component containing v_j. If $C_i^{q-1} \neq C_j^{q-1}$ and $w(o_q) \leq MInt(C_i^{q-1}, C_j^{q-1})$ then S^q is obtained from S^{q-1} by merging C_i^{q-1} and C_j^{q-1}. Otherwise $S^q = S^{q-1}$.
4. Return $S = S^m$.

http://blog.csdn.net/surgewong

相关链接：https://blog.csdn.net/surgewong/article/details/39008861

RCNN-coordinate regression



每一个$d_*(P)$【*表示x，y，w，h】都是一个AlexNet CNN网络Pool5层特征$\phi_5(P)$的线性函数，即$d_*(P) = w_*^T\phi_5(P)$，这里w_*^T就是所需要学习的回归参数。损失函数即为：

$$\hat{G}_x = P_w d_x(P) + P_x$$

$$\hat{G}_y = P_h d_y(P) + P_y$$

$$\hat{G}_w = P_w exp(d_w(P))$$

$$\hat{G}_h = P_h exp(d_h(P))$$

$$Loss = argmin \sum_{i=0}^{N} (t_*^i - \hat{w}_*^T \phi_5(P^i))^2 + \lambda ||\hat{w}_*||^2 \tag{5}$$

损失函数中加入正则项$\lambda||\hat{w}_*||^2$ 是为了避免归回参数w_*^T过大。其中，回归目标t_*由训练输入对(P, G)按下式计算得来：

$$t_x = (G_x - P_x)/P_w \tag{6}$$

$$t_y = (G_y - P_y)/P_h \tag{7}$$

$$t_w = log(G_w/P_w) \tag{8}$$

$$t_h = log(G_h/P_h) \tag{9}$$

SPP-NET

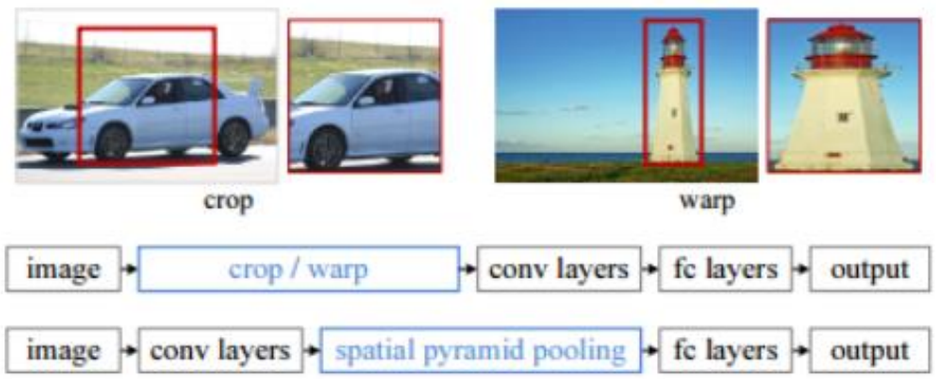


Figure 1: Top: cropping or warping to fit a fixed size. Middle: a conventional CNN. Bottom: our spatial pyramid pooling network structure.

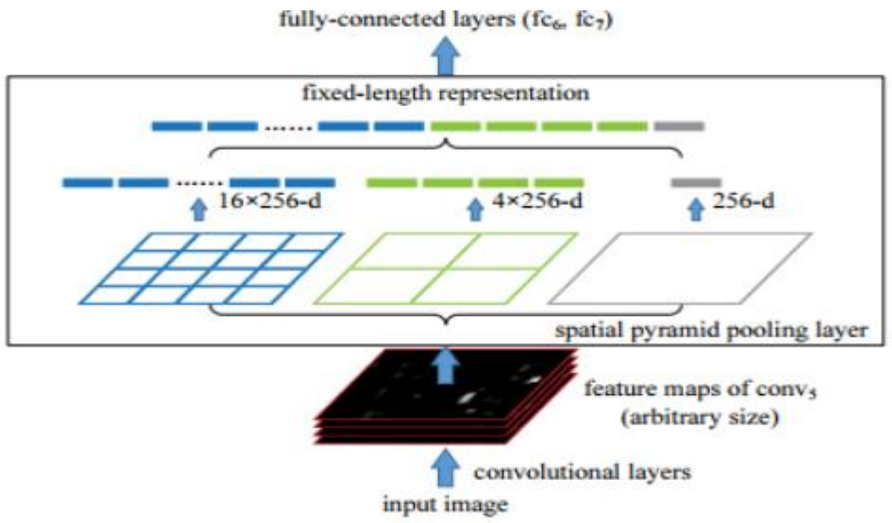


Figure 3: A network structure with a **spatial pyramid pooling layer**. Here 256 is the filter number of the $conv_5$ layer, and $conv_5$ is the last convolutional layer.

Why the spp cannot be backpropagated?

The root cause is that back-propagation through the SPP layer is highly inefficient when each training sample (i.e. RoI) comes from a different image, which is exactly how R-CNN and SPPnet networks are trained. The inefficiencystems from the fact that each RoI may have a very large receptive field, often spanning the entire input image. Since the forward pass must process the entire receptive field, the training inputs are large (often the entire image).

RCNN and SPP-NET's drawbacks

基于区域的卷积网络方法（RCNN）通过使用深层的ConvNet来分类对象建议框，实现了很棒的检测对象精度。 然而，R-CNN的缺点也很明显：

1. Training is a multi-stage pipeline 训练的时候，R-CNN首先使用log loss 在对象建议框（object proposals ）上微调ConvNet ，然后提取特征，之后用SVM分类器，最后再进行bounding-box regressors学习。
2. Training is expensive in space and time 在每张图片中对对象建议框（object proposals ）进行提取来对SVM和bounding-box regressor作训练，还要写进磁盘中。训练时间和空间开销很大。
3. Object detection is slow 对象检测很慢，即测试时间开销大。

RCNN很慢的原因主要是因为没有共享计算。SPP-net就通过共享计算提高了速度，它在ConvNet的最后一个卷积层才提取proposal，但是依然有不足之处。和R-CNN一样，它的训练要经过多个阶段，特征也要存在磁盘中，另外，SPP中的微调只更新spp层后面的全连接层，因为卷积特征是线下计算的，从而无法再微调阶段反向传播误差。对很深的网络这样肯定是不行的。

Fast Rcnn

&创新点：

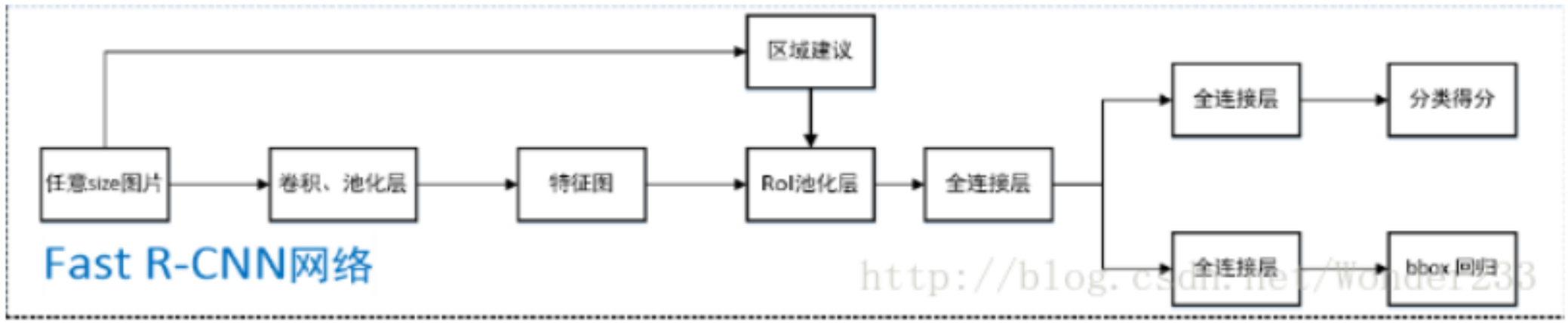
规避R-CNN中冗余的特征提取操作，只对整张图像全区域进行一次特征提取；用RoI pooling层取代最后一层max pooling层，同时引入建议框信息，提取相应建议框特征；

Fast R-CNN网络末尾采用并行的不同的全连接层，可同时输出分类结果和窗口回归结果，实现了end-to-end的多任务训练【建议框提取除外】，也不需要额外的特征存储空间【R-CNN中这部分特征是供SVM和Bounding-box regression进行训练的】；

采用SVD对Fast R-CNN网络末尾并行的全连接层进行分解，减少计算复杂度，加快检测速度。

相关链接：https://blog.csdn.net/WoPawn/article/details/52463853?locationNum=7

Fast Rcnn



&测试过程：

（1）任意size图片输入CNN网络，经过若干卷积层与池化层，得到特征图；在任意size图片上采用selective search算法提取约2k个建议框；

（2）根据原图中建议框到特征图映射关系，在特征图中找到每个建议框对应的特征框【深度和特征图一致】，并在RoI池化层中将每个特征框池化到H×W【VGG-16网络是7×7】的size；

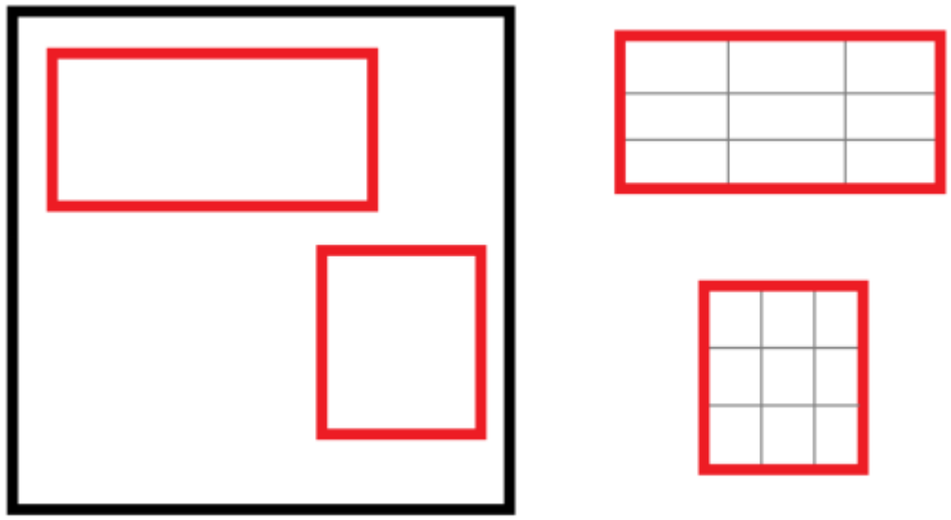
（3）固定H×W【VGG-16网络是7×7】大小的特征框经过全连接层得到固定大小的特征向量；

（4）第4步所得特征向量经由各自的全连接层【由SVD分解实现】，分别得到两个输出向量：一个是softmax的分类得分，一个是Bounding-box窗口回归；利用窗口得分分别对每一类物体进行非极大值抑制剔除重叠建议框，最终得到每个类别中回归修正后的得分最高的窗口。

Fast Rcnn-ROI pooling

&原理：

首先假设建议框对应特征图中的特征框大小为h×w，将其划分H×W个子窗口，每个子窗口大小为h/H×w/W，然后对每个子窗口采用max pooling下采样操作，每个子窗口只取一个最大值，则特征框最终池化为H×W的size【特征框各深度同理】，这将各个大小不一的特征框转化为大小统一的数据输入下一层。



why the roi pooling layer can be propogated but the spp cannot?

We propose a more efficient training method that takes advantage of feature sharing during training. In Fast RCNN training, stochastic gradient descent (SGD) minibatches are sampled hierarchically, first by sampling N images and then by sampling R=N RoIs from each image.

Critically, RoIs from the same image share computation and memory in the forward and backward passes. Making N small decreases mini-batch computation. For example, when using N = 2 and R = 128, the proposed training scheme is roughly 64× faster than sampling one RoI from 128 different images (i.e., the R-CNN and SPPnet strategy).

Fast Rcnn loss

当预测值与目标值相差很大时，梯度容易爆炸，因为梯度里包含了$x-t$. 所以rgb在Fast RCNN里提出了SmoothL1Loss.

$$L_{\text{loc}}(t^u, v) = \sum_{i \in \{x,y,w,h\}} \text{smooth}_{L_1}(t_i^u - v_i), \qquad (2)$$

in which

$$\text{smooth}_{L_1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise,} \end{cases} \qquad (3)$$

http://blog.csdn.net/weixin_35653315

当差值太大时，原先L2梯度里的$x-t$被替换成了± 1，这样就避免了梯度爆炸，也就是它更加健壮.

Fast-RCNN为了使用SmoothL1Loss定义了一个新的layer，它的实现更general:

$$x_i = w_{in}(t_i - v_i), i \in \{x, y, w, h\}$$

$$smooth_{L_1} = \begin{cases} 0.5x^2\sigma^2, & |x\sigma| < 1 \\ |x| - \frac{0.5}{\sigma^2}, & otherwise \end{cases}$$

$$SmoothL1Loss = w_{out} \sum_{i \in \{x,y,w,h\}} smooth_{L_1}(x_i)$$

- w_{in}可用于指定哪些regression结果参与loss的计算(Fast RCNN里的λ[u ≥ 1], Faster RCNN里的P_*).
- w_{out}可用于normalization.

由真实分类u对应的概率决定：

$$L_{cls}(p, u) = -logp_u$$

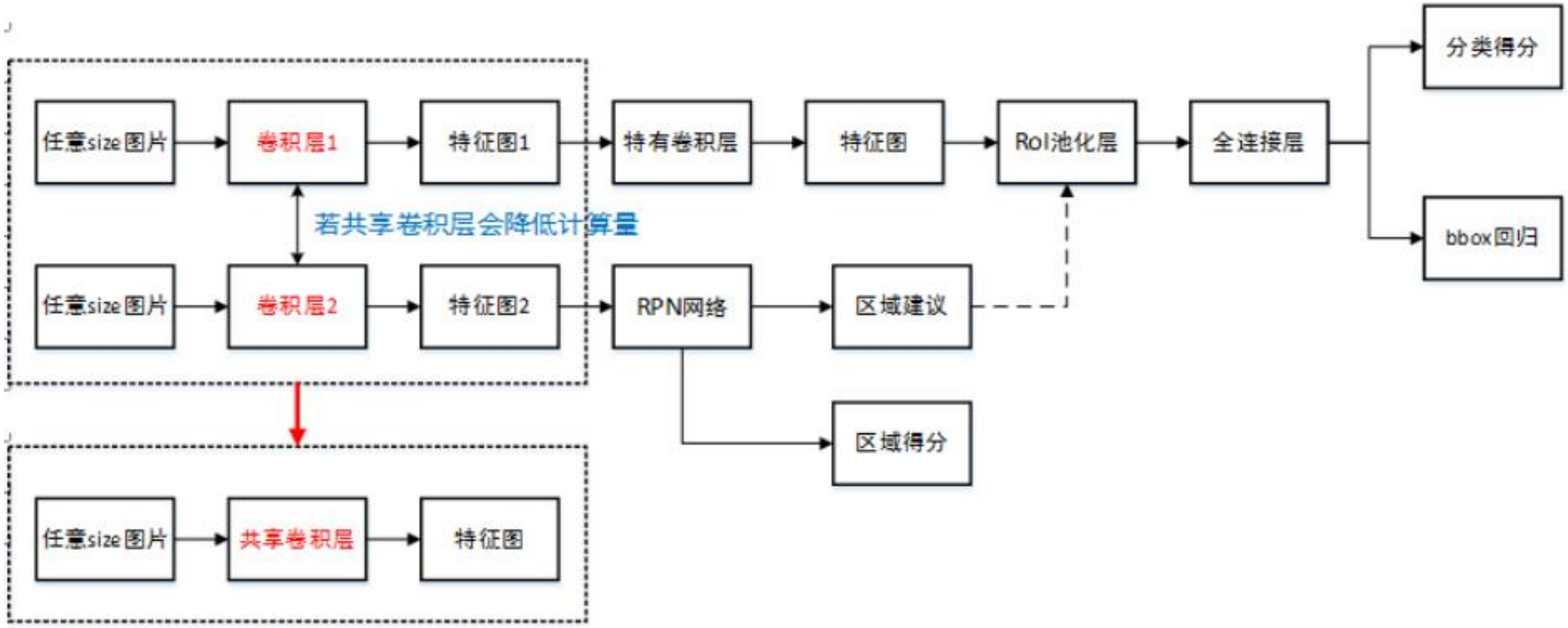
Fast R-CNN微调阶段总的损失函数为：

$$L(p, u, t^u, v) = L_{cls}(p, u) + \lambda[u \geq 1]L_{loc}(t^u, v)$$

$$[u \geq 1] = \begin{cases} 1, & u > 1 \\ 0, & \text{otherwise} \end{cases}$$

相关链接：https://blog.csdn.net/weixin_35653315/article/details/54571681

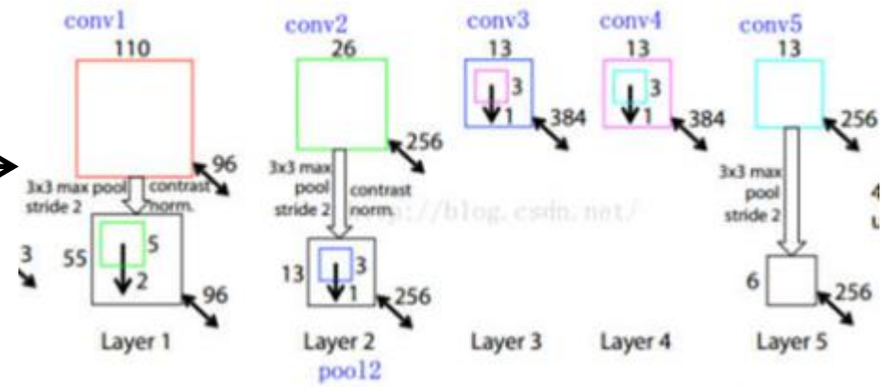
Faster Rcnn



相关链接：https://blog.csdn.net/wopawn/article/details/52223282

Faster Rcnn-RPN

resize → image →



feature map → k scores

feature map → 4k coordinates

&training:

input:resized image

output:selected pos and neg(total in 256 per image) in feature map

Faster Rcnn-RPN Loss

$$L(\{p_i\},\{t_i\}) = \frac{1}{N_{cls}}\sum_i L_{cls}(p_i,p_i^*) + \lambda\frac{1}{N_{reg}}\sum_i p_i^* L_{reg}(t_i,t_i^*)$$

其中，i表示一个mini-batch中某个anchor的下标，p_i表示anchor i预测为物体的概率；当anchor为正样本时，$p_i^* = 1$，当anchor为负样本时$p_i^* = 0$，由此可以看出回归损失项仅在anchor为正样本情况下才被激活；

t_i表示正样本anchor到预测区域的4个平移缩放参数【以anchor为基准的变换】；t_i^*表示正样本anchor到Ground Truth的4个平移缩放参数【以anchor为基准的变换】；

分类损失函数L_{cls} 是一个二值【是物体或者不是物体】分类器，
$L_{cls}(p_i,\ p_i^*) = -log[p_i^* p_i + (1-p_i^*)(1-p_i)]$；

归回损失函数$L_{reg}(t_i,t_i^*) = R(t_i - t_i^*)$【两种变换之差越小越好】，R函数定义如下：

$$smooth_{L1}(x) = \begin{cases} 0.5x^2, & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

λ参数用来权衡分类损失L_{cls} 和回归损失L_{reg}，默认值$\lambda = 10$【文中实验表明 λ从1变化到100对mAP影响不超过1%】；

N_{cls}和N_{reg}分别用来标准化分类损失项L_{cls} 和回归损失项L_{reg}，默认用mini-batch size=256设置N_{cls}，用anchor位置数目~2400初始化N_{reg}，文中也说明标准化操作并不是必须的，可以简化省略。

Faster Rcnn

Faster 过程阐述：

RPN: RPN训练 → RPN预测 → 针对Feature map上的每个点都有一个score、coor

Fast： 网络输入为图片 + ROI

输出为类别 + coor

proposal： RPN网络得到的感 Region proposal → NMS → 300 RPOS.

→ RP + ground truth = ROI.

人为的为标签做ROI pooling.

Faster Rcnn

SGD mini-batch采样方式：

同Fast R-CNN网络，采取”image-centric”方式采样，即采用层次采样，先对图像取样，再对anchors取样，同一图像的anchors共享计算和内存。每个mini-batch包含从一张图中随机提取的256个anchors，正负样本比例为1:1【当然可以对一张图所有anchors进行优化，但由于负样本过多最终模型会对正样本预测准确率很低】来计算一个mini-batch的损失函数，如果一张图中不够128个正样本，拿负样本补凑齐。

理清文中anchors的数目：

文中提到对于1000×600的一张图像，大约有20000(~60×40×9)个anchors，忽略超出边界的anchors剩下6000个anchors，利用非极大值抑制去掉重叠区域，剩2000个区域建议用于训练；

测试时在2000个区域建议中选择Top-N【文中为300】个区域建议用于Fast R-CNN检测。

Faster Rcnn

&文中提到了三种共享特征网络的训练方式?

① 交替训练

训练RPN，得到的区域建议来训练Fast R-CNN网络进行微调；此时网络用来初始化RPN网络，迭代此过程【文中所有实验采用】；

② 近似联合训练

如上图所示，合并两个网络进行训练，前向计算产生的区域建议被固定以训练Fast R-CNN；反向计算到共享卷积层时RPN网络损失和Fast R-CNN网络损失叠加进行优化，但此时把区域建议【Fast R-CNN输入，需要计算梯度并更新】当成固定值看待，忽视了Fast R-CNN一个输入：区域建议的导数，则无法更新训练，所以称之为近似联合训练。实验发现，这种方法得到和交替训练相近的结果，还能减少20%~25%的训练时间，公开的python代码中使用这种方法；

③ 联合训练

需要RoI池化层对区域建议可微，需要RoI变形层实现，具体请参考这片paper：Instance-aware Semantic Segmentation via Multi-task Network Cascades。

QUESTION

江南大学
JIANGNAN UNIVERSITY

Q & A